KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

EVENT REGISTRATION AND PARTICIPANT MANAGEMENT SYSTEM

Submitted by

NIVETHA A
25BIT036

Under the Guidance of

DHILEEBAN A
Technical Trainer
Technical Department, KG College of Arts and Science

Academic Year: 2025 - 2026
APRIL 2026

Semester: 2

DECLARATION

I hereby declare that the project entitled "Event Registration and Participant Management System" submitted in partial fulfillment of the requirements for the course [Java Programming] is a record of my original work carried out under the guidance of[DHILEEBAN A], Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

NIVETHA A

(Your Name)

CERTIFICATE

This is to certify that the project entitled "Event Registration and Participant Management System" is a bona fide work carried out by [NIVETHA A] [C1-B.Sc.IT], [INFORMATION TECHNOLOGY], KG College of Arts & Science, in partial fulfillment of the requirements for the course [Java Programming].

This project has been completed under my supervision and guidance during the academic period April 2026.

Place: _____
Date: _____

Submitted for the Viva-Voce Examination held on _____

Guide	Head of the Department
_____	_____

TABLE OF CONTENTS

S.No.	Topics	Page No.
1	INTRODUCTION	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	PROGRAM ANALYSIS	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	SYSTEM DESIGN	
3.1	Modules	
4	METHODOLOGY	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	SOURCE CODE	
6	INPUT AND OUTPUT SCREENS	
7	DISCUSSION AND CONCLUSION	
7.1	Discussion	
7.2	Conclusion	
8	REFERENCES	

ABSTRACT

The Event Registration and Participant Management System is a Java-based application designed to help event organizers and participants manage event registrations efficiently and receive timely event notifications. In many event management offices, participant records are maintained manually on paper or in spreadsheets, which can lead to missed registration deadlines, data errors, and poor attendee experience.

This project automates the complete lifecycle of an insurance policy — from policy registration and customer enrollment to premium calculation, payment tracking, and reminder scheduling. The system allows staff to add new participants, search existing registrations, calculate event fees, record payments, and generate notification alerts for upcoming or past-due registrations.

The application is developed using Core Java programming concepts including classes, objects, inheritance, encapsulation, methods, loops, conditional statements, file handling, and exception handling. The system can be implemented as a console-based application or enhanced with a Java Swing graphical user interface (GUI).

The primary objective is to reduce missed registrations, eliminate manual paperwork, improve record management, and provide timely notifications to participants. This project demonstrates the practical implementation of object-oriented programming concepts and file handling in Java through a real-world event management application.

1. INTRODUCTION

The event management industry plays a critical role in organizing conferences, workshops, and community gatherings. Event organizers manage hundreds of registrations, each with unique attendance details, fee schedules, and event dates. Ensuring that participants complete their registration on time is essential to maintaining accurate headcounts and avoiding cancellations.

Traditionally, event offices maintain participant records manually using paper-based systems or basic spreadsheets. These methods are highly prone to human error, make it difficult to track pending registrations, and often result in missed notification alerts to participants. This leads to no-shows, revenue losses, and poor attendee satisfaction.

The Event Registration and Participant Management System is a Java-based application that automates all key aspects of event registration management. The system provides a centralized platform where organizers can register new events, manage participant data, calculate registration fees, record payments, and automatically generate notifications for upcoming or pending registrations.

This project is developed using Core Java programming concepts such as classes, objects, methods, inheritance, file handling, and exception handling. The application simplifies event operations, improves accuracy, and ensures that no registration deadline goes unnoticed.

1.1 Objective

The main objective of the Event Registration and Participant Management System is to develop a Java-based application that automates policy management and ensures timely premium collection through automated reminders.

The specific objectives of this project are:

- To design and develop a Java-based event registration and participant management system.
- To automate the registration and storage of events and participant data.
- To calculate registration fees based on event type, category, and participant count.
- To track registration payment history and identify pending registrations.
- To generate automated registration notifications for upcoming and overdue registrations.
- To provide a search and report facility to retrieve participant registration information quickly.
- To demonstrate the implementation of Core Java concepts including OOP, file handling, and exception handling.

- To reduce manual effort and improve efficiency in event management operations.

1.2 Overview

The Event Registration and Participant Management System is a menu-driven application developed in Java that provides a complete solution for managing insurance policies and premium reminders. The system maintains records of policyholders, their policy details, and payment schedules.

When a new participant registers for an event, their details are stored in the system along with the event type, category, capacity, and payment frequency. The system calculates the registration fee automatically and generates a payment schedule. As registration deadlines approach, the system flags pending registrations and displays notification alerts.

The system allows event staff to record registration payments, update participant status, search for specific registrations, and view complete payment histories. It provides a structured and organized method of managing all event-related data that can easily replace traditional paper-based systems.

1.3 Features

The Event Registration and Participant Management System includes the following key features:

- Event Registration -- Register new events with complete organizer and event details.
- Participant Management -- Store and manage participant personal information and contact details.
- Premium Calculation -- Automatically calculate registration fees based on event type, category, and participant count.
- Payment Recording -- Record and track registration payments with date and amount details.
- Participant Management System -- Generate alerts for policies with upcoming or overdue premium payments.
- Registration Search -- Search and retrieve specific registration details using registration ID or participant name.
- Payment History -- View complete payment history for any registration.
- Registration Status Management -- Track whether registrations are confirmed, pending, cancelled, or waitlisted.
- Report Generation -- Generate summary reports of all events and participant statuses.
- Data Persistence -- Save and load registration data using Java file handling.

2. PROGRAM ANALYSIS

Program analysis is a fundamental phase in software development where the existing system is studied in detail, problems are identified, and the requirements for the new system are defined. This phase involves understanding the domain of event management operations and identifying the key data and processes involved.

For the Event Registration and Participant Management System, the analysis focuses on understanding how insurance policies are managed, how premiums are calculated, and how reminders are communicated to policyholders.

2.1 Problem Definition

Event organizers managing a large number of registrations face significant challenges when relying on manual systems for participant tracking and fee collection. The following problems are commonly encountered in traditional event management:

- Missed registration deadlines due to lack of automated notifications.
- Event cancellations due to delayed communication with participants.
- Manual calculation errors in registration fees and dues.
- Difficulty in retrieving specific participant details from large paper records.
- No centralized record of payment history for each participant.
- Inability to quickly identify overdue or cancelled registrations.
- High administrative workload on event staff for routine tasks.
- Loss of important documents due to poor physical record management.

These problems directly impact the efficiency of the event organization and lead to financial losses due to missed registration collections. A computerized system is essential to solve these challenges.

2.2 Project Overview

The Event Registration and Participant Management System is a Java-based application designed to automate the management of event registrations and participant notifications. The system maintains a database of participants and their associated events.

Each registration record contains details such as registration ID, participant name, contact information, event type (conference, workshop, seminar, cultural), event date, venue, registration category (individual, group, VIP), calculated registration fee, payment history, and registration status.

The system performs the following key functions:

- Registering new participants and events in the system.
- Automatically calculating registration fees based on event parameters.
- Generating and displaying registration payment schedules.
- Tracking registration payment records.
- Identifying and displaying upcoming deadlines and overdue registration reminders.
- Searching and displaying participant and event details on request.
- Updating registration status based on payment history.
- Saving all data to files for future sessions.

2.3 Scopes and Limitations

Scope

The scope of the Event Registration and Participant Management System includes:

- Managing all major types of events -- conferences, workshops, seminars, and cultural programs.
- Supporting multiple registration categories -- individual, group, student, and VIP.
- Maintaining complete payment history and registration lifecycle records.
- Generating registration reminders for events scheduled within a configurable number of days.
- Searching registrations by registration ID, participant name, or event type.
- Generating summary reports for all active registrations and pending payments.
- Persisting data using Java file I/O for use across multiple sessions.

Limitations

- The current version does not include a graphical user interface; it operates as a console-based application.
- The system does not integrate with a relational database; data is stored using file handling.
- Email or SMS-based automatic notifications are not implemented in the basic version.
- The system does not support online payment gateway integration.
- Multi-user access control and role-based permissions are not included in this version.

3. SYSTEM DESIGN

System design is the phase where the architecture and components of the application are planned before development begins. This stage defines how the system will be structured, what modules will be created, how data will flow through the system, and how each component will interact with others.

The Event Registration and Participant Management System is designed using an object-oriented approach. The system is divided into logical modules, each responsible for a specific functional area. The design ensures modularity, maintainability, and ease of future enhancement.

3.1 Modules

The system is composed of the following major modules:

1. Participant Registration Module

This module handles the registration of new participants and their events. It collects all necessary information including personal details, event type, category, event date, and duration. Data validation is performed before storing the record.

Key operations: Add new participant, assign unique registration ID, store participant contact details.

2. Fee Calculation Module

This module calculates the registration fee based on the event parameters entered during registration. Different formulas are applied depending on the type of event.

Sample fee calculation formulas:

- Conference: Fee = (Base Fee x Category Multiplier) / (Duration in Days)
- Workshop: Fee = Base Amount + (Participant Count x Per Person Rate)
- Seminar: Fee = Event Cost x Registration Category Percentage
- Cultural Program: Fee = Venue Cost x Participant Percentage

3. Payment Tracking Module

This module records all registration payments made by participants. Each payment entry includes the registration ID, payment date, amount paid, and payment mode. The module maintains a complete payment history for every registration and updates the registration status accordingly.

4. Notification Module

This is the core feature of the system. The module scans all active registrations and compares their event dates with the current date. It generates notification alerts for:

- Registrations with events scheduled within the next 30 days (upcoming notifications).
- Registrations with overdue payments (immediate alert).
- Registrations under cancellation grace period (last warning).

5. Participant Search and Display Module

This module allows staff to search for specific registrations using different search criteria such as registration ID, participant name, or event type. It displays all relevant registration details including current status and payment history.

6. Report Generation Module

This module generates summary and detailed reports of all registrations, including active participant lists, event schedules, payment summaries, and cancelled registration reports.

7. File Handling Module

This module manages the persistent storage of all registration and payment data using Java file I/O. It reads data at application startup and saves updated data when the application closes or when records are modified.

8. Main Menu Module

This module provides the main interface of the application. It displays a numbered menu to the user and routes input to the appropriate functional module using control flow statements.

4. METHODOLOGY

Methodology describes the structured approach followed to plan, develop, test, and document the Event Registration and Participant Management System. A phased software development methodology was adopted to ensure each stage of development was completed systematically before proceeding to the next.

4.1 Problem Analysis

In this phase, the requirements of the event registration management domain were carefully studied. Interviews and observations of typical event office operations were used to understand data requirements and workflow.

Key problems identified during analysis include missed registration notifications, inaccurate fee calculations, poor record management, and difficulty in tracking pending registrations. Based on these findings, the system requirements were documented to guide the design and development phases.

4.2 System Design

Based on the requirements identified during analysis, the system architecture was designed using object-oriented principles. The following Java classes were designed:

- Event (com.event.model) -- Stores event attributes: eventId, eventName, registrationFee, eventDate. Provides getters and a toString() method.
- EventManager (com.event.service) -- Manages a list of Event objects. Provides addEvent(), displayAllEvents(), and checkNotifications() to flag events scheduled within the next 7 days.
- EventApp (com.event.main) -- Entry point of the application. Collects event details from the user via Scanner, creates an Event object, adds it to EventManager, and triggers display and notification checks.

4.3 Program Development

The application was developed using Core Java. The following development steps were followed:

1. Setting up the Java development environment (JDK and IDE/text editor).
2. Creating the class structure based on the system design.
3. Implementing participant registration and data validation logic.
4. Developing the fee calculation module with event-type-specific formulas.
5. Building the payment recording and tracking module.
6. Implementing the notification system with date comparison logic.
7. Developing the search and display functionality.
8. Adding file handling for data persistence across sessions.
9. Building the main menu and integrating all modules.
10. Testing and debugging the complete application.

4.4 Testing and Validation

Comprehensive testing was carried out to verify the correctness and reliability of the system. The following test cases were designed and executed:

No.	Test Case	Expected Result	Status
1	Register new participant with valid data	Registration saved successfully	Pass
2	Register participant with missing fields	Validation error displayed	Pass
3	Calculate fee for conference registration	Correct fee computed	Pass
4	Record registration payment	Payment stored and status updated	Pass
5	Search registration by registration ID	Correct registration details displayed	Pass
6	Generate notifications for overdue registrations	Overdue list displayed correctly	Pass
7	Search with invalid registration ID	'Policy not found' message shown	Pass
8	Save and reload data from file	All records loaded correctly	Pass

4.5 Result and Documentation

After successful testing, the system was found to meet all defined requirements. All modules function correctly and produce expected outputs. The system accurately calculates registration fees, records payments, and generates timely notifications. Documentation was prepared covering the system design, source code, and sample input/output screens.

4.6 Algorithm

Algorithm for Event Registration and Participant Management System:

11. Start the program.
12. Load existing registration and event data from file.
13. Display the main menu.
14. Accept user menu choice.
15. If choice is 1: Accept participant details, calculate registration fee, assign registration ID, save record.
16. If choice is 2: Accept registration ID, record payment amount and date, update registration status.
17. If choice is 3: Scan all registrations, compare event date with current date, display notifications for events within 30 days or overdue.

18. If choice is 4: Accept search input, locate and display matching registration details.
19. If choice is 5: Generate and display all registration report.
20. If choice is 6: Save all data to file and exit the program.
21. Return to step 3 if choice is not 6.
22. End.

4.7 Flowchart

The following represents the program flow of the Event Registration and Participant Management System:

Start

↓

Load Data from File

↓

Display Main Menu

↓

Accept User Choice

↓

```
[1] Register Policy   [2] Record Payment   [3] View Reminders   [4] Search
Policy   [5] Report   [6] Exit
```

↓

Execute Selected Module → Display Result

↓

Return to Menu? (Yes → Menu) / (No → Save & Exit)

5. SOURCE CODE

Participant.java

```
package com.event.model;

public class Participant {

    private int id;
    private String name;
    private String email;
    private String eventName;

    public Participant(int id, String name, String email, String eventName) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.eventName = eventName;
    }

    public void display() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Email: " + email);
        System.out.println("Event: " + eventName);
        System.out.println("----------------------");
    }
}
```

ParticipantService.java

```
package com.event.service;

import java.util.ArrayList;
import com.event.model.Participant;

public class ParticipantService {

    ArrayList<Participant> list = new ArrayList<>();


    public void addParticipant(Participant p) {
        list.add(p);
        System.out.println("Participant Registered Successfully!");
    }


    public void showParticipants() {
        System.out.println("\n--- Participant List ---");
        for (Participant p : list) {
```

```
            p.display();
        }
    }
}
```

MainApp.java

```
package com.event.main;

import com.event.model.Participant;
import com.event.service.ParticipantService;

public class MainApp {
    public static void main(String[] args) {

        ParticipantService service = new ParticipantService();


        Participant p1 = new Participant(1, "Nivetha", "nivetha@gmail.com", "Tech Fest");
        Participant p2 = new Participant(2, "Harsha", "harsha@gmail.com", "Workshop");

        service.addParticipant(p1);
        service.addParticipant(p2);


        service.showParticipants();
    }
}
```

6. INPUT AND OUTPUT SCREENS

Participant Registered Successfully!

Participant Registered Successfully!

--- Participant List ---

ID: 1

Name: Nivetha

Email: nivetha@gmail.com

Event: Tech Fest

ID: 2

Name: Harsha

Email: harsha@gmail.com

Event: Workshop

----

7. DISCUSSION AND CONCLUSION

7.1 Discussion

The Event Registration and Participant Management System was developed to address the real-world challenges faced by insurance offices that rely on manual processes for policy tracking and premium collection. The system successfully automates all key aspects of insurance policy management, from registration to reminder generation.

During development, Core Java programming concepts were applied extensively. Classes and objects were used to model real-world entities such as policies and policyholders. Inheritance and encapsulation helped organize the code into modular and reusable components. The LocalDate class from Java's java.time package was used to implement precise date calculations for premium due dates and reminder generation.

The notification module proved to be the most impactful component of the system. By comparing the event date with the current date, the system reliably identifies upcoming events and overdue

payments. This feature alone can significantly reduce no-shows and improve revenue collection for event organizations.

Testing confirmed that the system handles various input scenarios correctly, including edge cases such as invalid inputs, missing data, and boundary date conditions. Exception handling ensured the program does not crash unexpectedly during runtime.

The file handling module enables data persistence, allowing the system to retain all registration and payment records between sessions. This makes the system suitable for real-world deployment in small event management offices.

7.2 Conclusion

The Event Registration and Participant Management System successfully demonstrates the practical application of Core Java programming in developing a real-world business application for the insurance domain. The system fulfills all the stated objectives by providing automated policy management, premium calculation, payment tracking, and reminder generation capabilities.

The system reduces manual effort, eliminates calculation errors, and ensures timely communication of event dates to participants. This directly contributes to improved attendee satisfaction and reduced no-shows for event organizations.

This project helped in gaining hands-on experience with Java object-oriented programming, date handling, file I/O, exception handling, and modular program design. It demonstrates how Java programming can be applied to solve practical business problems in domains such as insurance, finance, and administration.

In the future, the system can be further enhanced by integrating a relational database (MySQL with JDBC), adding a graphical user interface using Java Swing or JavaFX, implementing automatic email or SMS notification alerts, adding multi-user access with role-based permissions, and enabling online registration payment integration.

8. REFERENCES

Books

- Herbert Schildt -- Java: The Complete Reference, 12th Edition, McGraw-Hill.
- Cay S. Horstmann -- Core Java Volume I -- Fundamentals, Pearson Education.
- Y. Daniel Liang -- Introduction to Java Programming, Comprehensive Version, Pearson.

Websites

- https://www.oracle.com/java -- Official Java Documentation
- https://www.geeksforgeeks.org -- Java Programming Concepts
- https://www.javatpoint.com -- Java Tutorials and Examples
- https://www.w3schools.com/java -- Java Reference and Examples
- https://docs.oracle.com/en/java/ -- Java SE API Documentation

Additional References

- Ministry of Corporate Affairs, Government of India -- www.mca.gov.in
- Java LocalDate Documentation -- https://docs.oracle.com/javase/8/docs/api/java/time/LocalDate.html